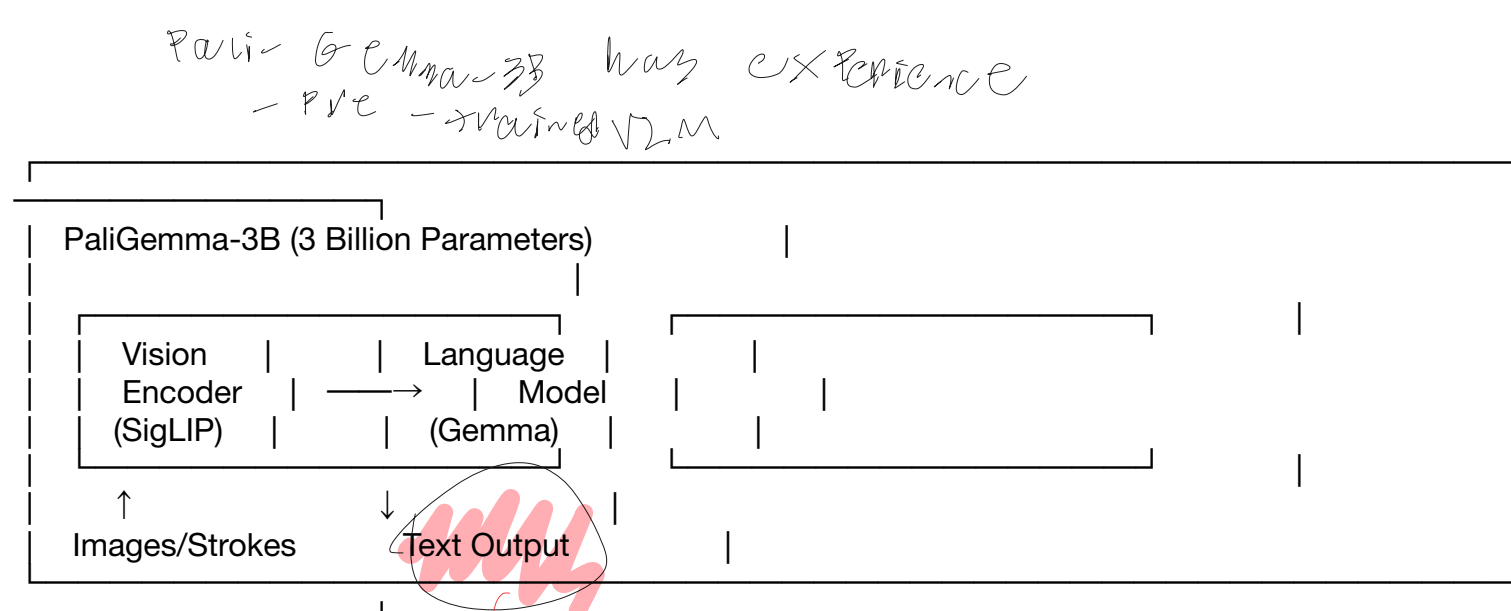




→ LaTeX?

3 billion params = 3 billion ways VLM make connections
- neurons in brain

Yeah let's implement weight snapshots so we know how model performs
at each epoch & if it converges

Let's use LoRA!

- try $r=16$

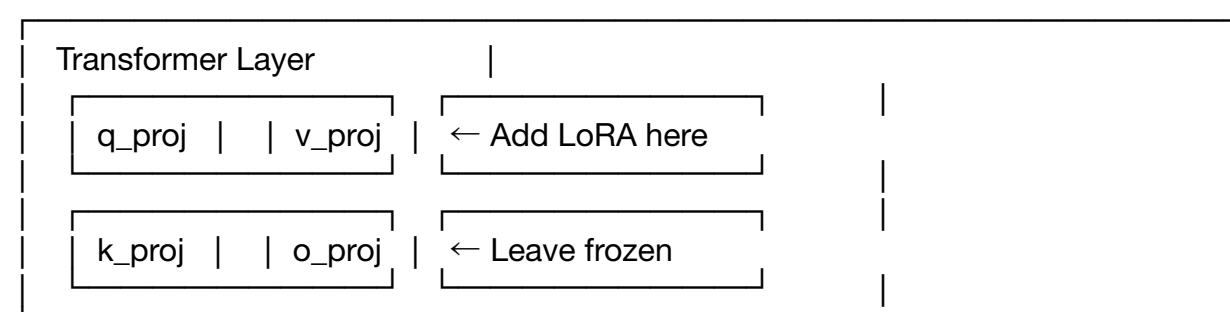
- Avoids overfitting

α - lower = less + base model noise

- α of 32 ✓

Add dropout to
prevent overfitting
- 0.05 - 0.1

Transformer has many layers. Which ones to adapt?



q_proj = Query projection (what to look for)
v_proj = Value projection (what information to extract)

Paper found these two are most important for adaptation

I am using NVIDIA A100 40GB
- High ram model

- implement highest batch size
for our GPU

- copy paper's padding & truncation
approach

CER Strategy

```
class CERMetric:
```

```
    def __init__(self):
```

```
        self.symbol_errors = {} # Track errors per symbol
```

```
        self.total_samples = 0
```

```
    def compute(self, predicted, ground_truth):
```

```
        # Calculate edit distance
```

```
        errors = levenshtein_distance(predicted, ground_truth)
```

```
        cer = errors / len(ground_truth)
```

```
        # Track which symbols failed
```

```
        for symbol in ground_truth:
```

```
            if symbol not in predicted:
```

```
                self.symbol_errors[symbol] = self.symbol_errors.get(symbol, 0) + 1
```

```
    return cer
```

```
    def get_worst_symbols(self, top_k=10):
```

```
        # Return symbols with most errors
```

```
        return sorted(self.symbol_errors.items(),
```

```
                        key=lambda x: x[1], reverse=True)[:top_k]
```